CABILLON JERXEL - ANSWERS

Pre-Lab Questions

a. Define operating system.
An operating system (OS) is system software that manages a computer's hardware and software resources and acts as an intermediary between the user (or applications) and the hardware. It handles tasks like memory management, process management, file systems, and I/O device control so that individual programs don't have to deal with hardware directly.

b. What are the different types of operating systems?

- **Batch OS** – executes jobs in batches with no direct user interaction.
- **Time-Sharing OS** – allows multiple users/processes to share CPU time via rapid switching, giving the illusion of simultaneous execution.
- **Distributed OS** – manages a group of independent computers as a single system, sharing resources over a network.
- **Network OS** – provides services to computers connected on a network (file sharing, security, etc.).
- **Real-Time OS (RTOS)** – guarantees processing within strict time constraints, used in embedded/critical systems.
- **Multiprocessing OS** – uses multiple CPUs within a single system to execute processes simultaneously.
- **Mobile OS** – designed specifically for smartphones/tablets (e.g., Android, iOS).

c. Define a process.
A process is a program in execution. It's an active entity that includes the program code, current activity (program counter, register values), and the resources it's using — such as memory and open files — as opposed to a program, which is just static code stored on disk.

d. What is CPU Scheduling?
CPU Scheduling is the method by which the operating system decides which process in the ready queue gets to use the CPU next. Its goal is to maximize CPU utilization and keep the system responsive and fair, since only one process can run on a CPU core at a time.

e. Define arrival time, burst time, waiting time, turnaround time.

- **Arrival time** – the time at which a process enters the ready queue, requesting CPU access.

CABILLON JERXEL - ANSWERS

- **Burst time** – the total amount of CPU time a process needs to complete its execution.
- **Waiting time** – the total time a process spends sitting in the ready queue waiting for the CPU (Turnaround time − Burst time).
- **Turnaround time** – the total time from when a process arrives to when it finishes execution (Completion time − Arrival time).

Post-Lab Questions

1. What is the real difficulty with the SJF CPU scheduling algorithm?
The main issue is that it requires knowing the burst time of each process in advance, which isn't realistic — the OS can only estimate it (e.g., using past behavior). SJF is also optimal for minimizing average waiting time but can cause **starvation**: a process with a long burst time can be repeatedly bumped by a stream of shorter jobs and wait indefinitely.

2. How can you determine which CPU algorithm is effective?
Effectiveness is judged by comparing metrics like average waiting time, average turnaround time, CPU utilization, throughput (processes completed per unit time), and response time across algorithms for the same set of processes. In my simulator, this can be observed directly — the same input processes give a lower (or equal) average waiting/turnaround time under SJF than FCFS, since SJF is provably optimal for average waiting time among non-preemptive algorithms.

3. How to compute the waiting time and burst time of every process?

- **Burst time** is given/estimated per process — it's the raw CPU time it needs, not calculated from other values.
- **Waiting time** is computed as: Waiting time = Turnaround time − Burst time, where Turnaround time = Completion time − Arrival time. In other words, waiting time is however long the process sat idle in the ready queue before it actually got the CPU.

4. How to compute the average waiting time and average burst time of every process?

- **Average waiting time** = (sum of waiting times of all processes) ÷ (number of processes)
- **Average burst time** = (sum of burst times of all processes) ÷ (number of processes)